SWEN30006
Software Design and Modelling

Modelling & Design in the Software Process

Reference: Larman Chapter 4, 8, 12, and 14

‘Le mieux est l’ennemi du bien (The best is the enemy of the good).”

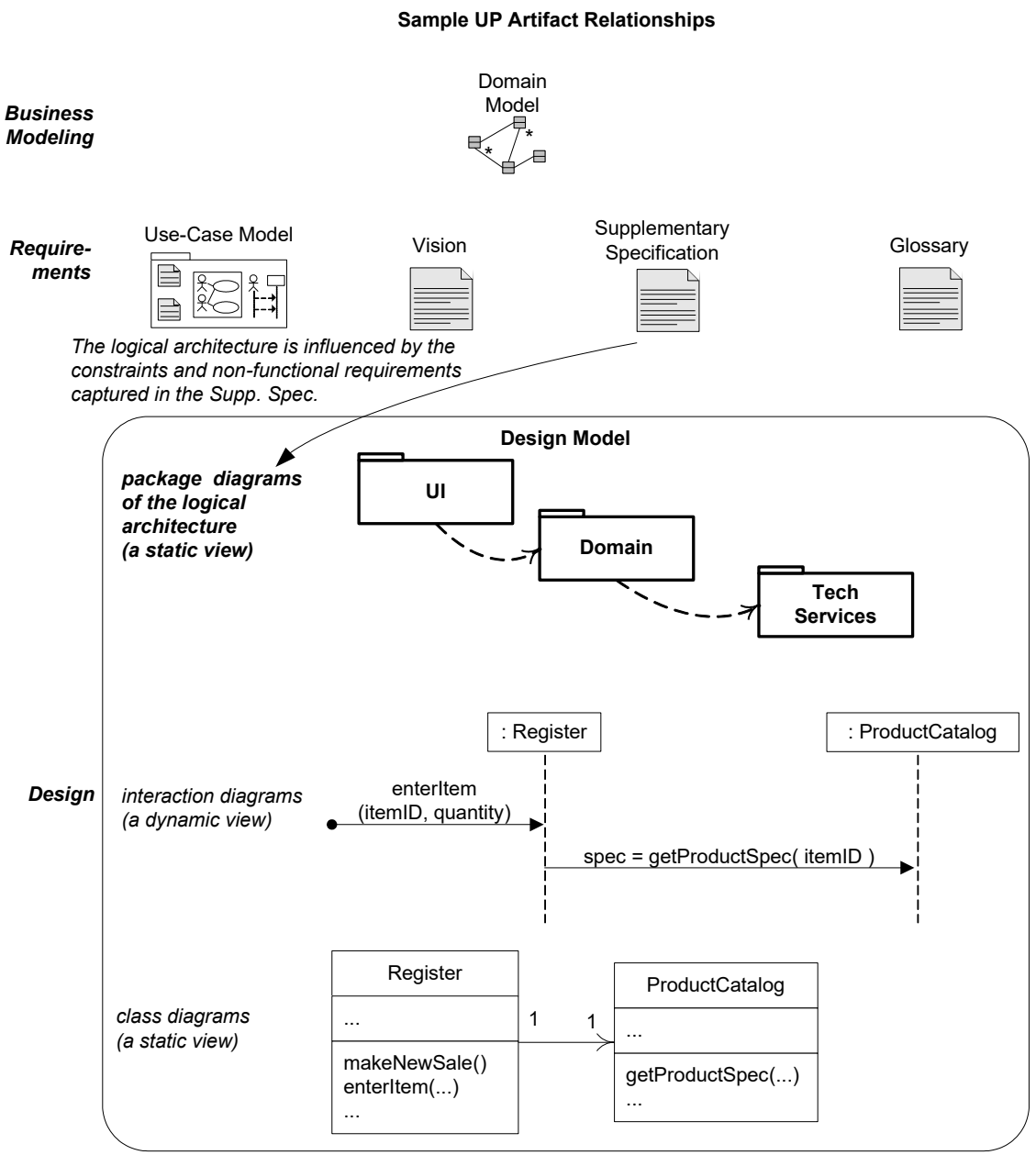
—Voltaire

Learning Objectives

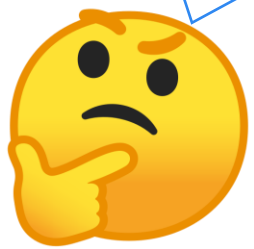
On completion of this topic, you should be able to:

- Relate the software modelling and design process to the software development process
- Outline and explain the key activities & artefacts of Inception and Elaboration

Recap

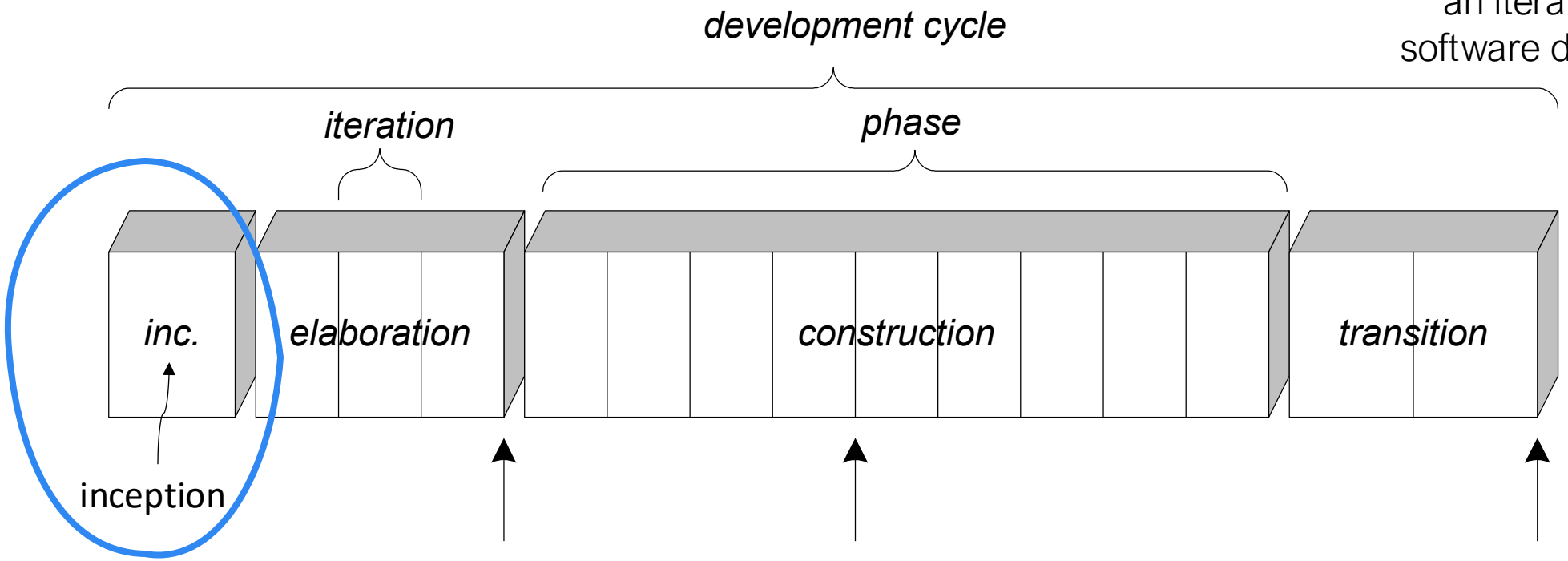


When do we design and create models in the software development process?



Schedule-Oriented Terms in UP

Unified Process (UP):
an iterative & incremental
software development process



milestone

An iteration end-point when some significant decision or evaluation occurs.

release

A stable executable subset of the final product. The end of each iteration is a minor release.

increment

The difference (delta) between the releases of 2 subsequent iterations.

final production release

At this point, the system is released for production use.

Inception

Definition: Inception is the initial short step to establish a common vision and basic scope for the project.

Focusing on answering questions like:

- What is the vision and business case for this project?
- Feasible?
- Buy and/or build?
- Rough unreliable range of cost: $10K–100K or $xM?
- Should we proceed or stop?
- *Do the stakeholders have basic agreement on the vision of the project, and is it worth investing in serious investigation?*

The UP is not the waterfall process, and the inception is *not* the time do all requirements or create believable estimates or plans. That happens during elaboration.

Outcome of Inception

- Common vision and basic scope for the project
- Creation of a business case (addressing cost)
- Analysis of ~10% of use cases
- Analysis of critical non-functional requirements
- Preparation of the development environment
- (Maybe) Prototypes: clarify requirements or technology questions
- Go or no go decision.

"In preparing for battle I have always found that plans are useless, but planning indispensable" - Eisenhower

Inception Artefacts (Some, Partial)

*must-haves

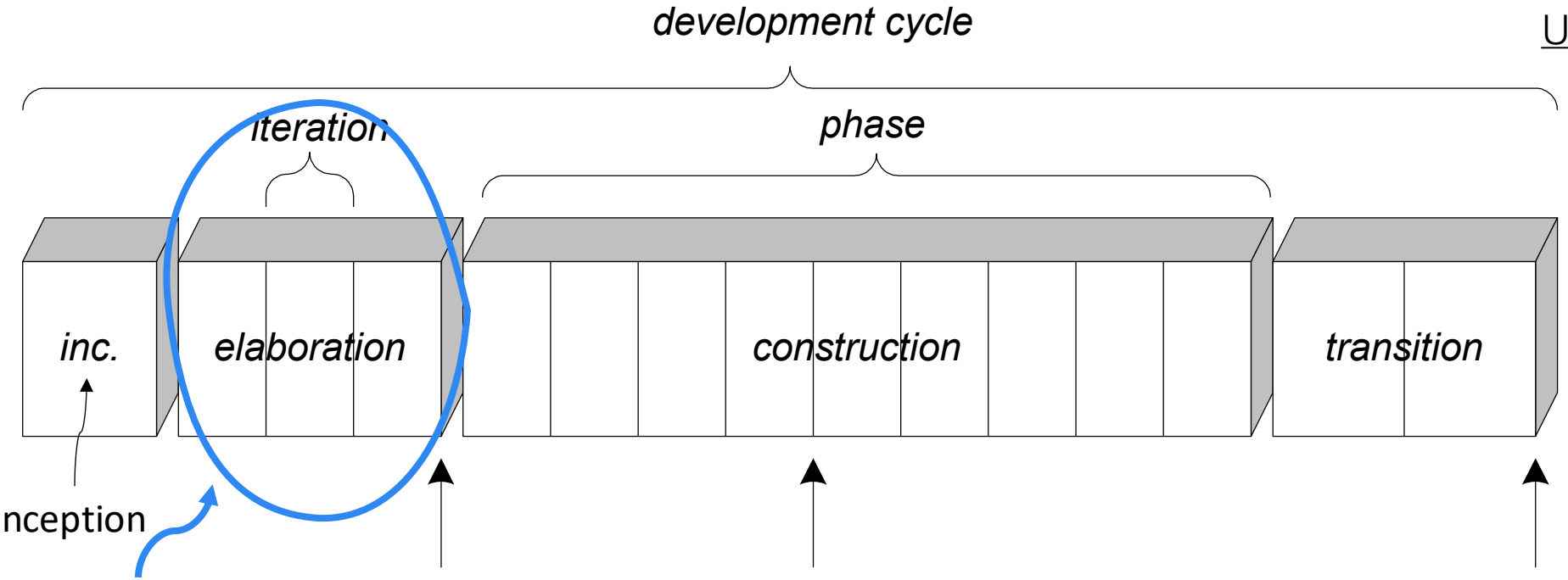
Artefact	Comment
Vision & Business Case*	Describes *high-level goals and constraints, business case*, and provides an executive summary.
Use-Case Model*	Describes functional requirements. During inception, *names of most use cases will be identified; ~10% of use cases analysed in detail*.
Supplementary Specification	Describes other requirements, mostly non-functional. During inception, useful to have some idea of ***key non-functional requirements with major impact on the architecture****.
Glossary	Key domain terminology, and data dictionary.
Risk List & Risk Management Plan	Describes risks (business, technical, resource, schedule) and ideas for their mitigation or response.
Prototypes & Proof-of-concepts	To clarify the vision and validate technical ideas.
Iteration Plan*	Describes what to do in the *first elaboration* iteration.
Phase Plan & Software Development Plan	Low-precision guess for elaboration phase duration and effort. Tools, people, education, and other resources.
Development Case	A description of the customized UP steps and artefacts for this project. In the UP, one always customizes for the project.

Not considered to be Inception if …

1. You took more than “a few” weeks for this phase.
2. You attempted to define most of the requirements.
3. You expect estimates or plans to be reliable.
4. You defined the architecture.
5. You planned the sequence of work: 1) define the requirements; 2) design the architecture; 3) implement.
6. You don’t produce a Business Case or Vision artefact.
7. You wrote all the use cases in detail.
8. You didn’t write any use cases in detail.

Schedule-Oriented Terms in UP

Unified Process



Design & modelling usually happen

milestone

An iteration end-point when some significant decision or evaluation occurs.

release

A stable executable subset of the final product. The end of each iteration is a minor release.

increment

The difference (delta) between the releases of 2 subsequent iterations.

final production release

At this point, the system is released for production use.

Elaboration

Definition: Elaboration is the initial series of iterations for building the core architecture, resolving the high-risk elements, defining most requirements, and estimating the overall schedule and resources.

After the elaboration:

- The core, risky software architecture is programmed and tested
  - Produce an executable architecture
- The majority of requirements are discovered and stabilized
- The major risks are mitigated or retired

Elaboration is *not* a do-the-whole-design phase or a phase when the models are <u>fully</u> developed in preparation for implementation in the construction step.

Elaboration Artifacts

Sample artifacts that may be started in elaboration

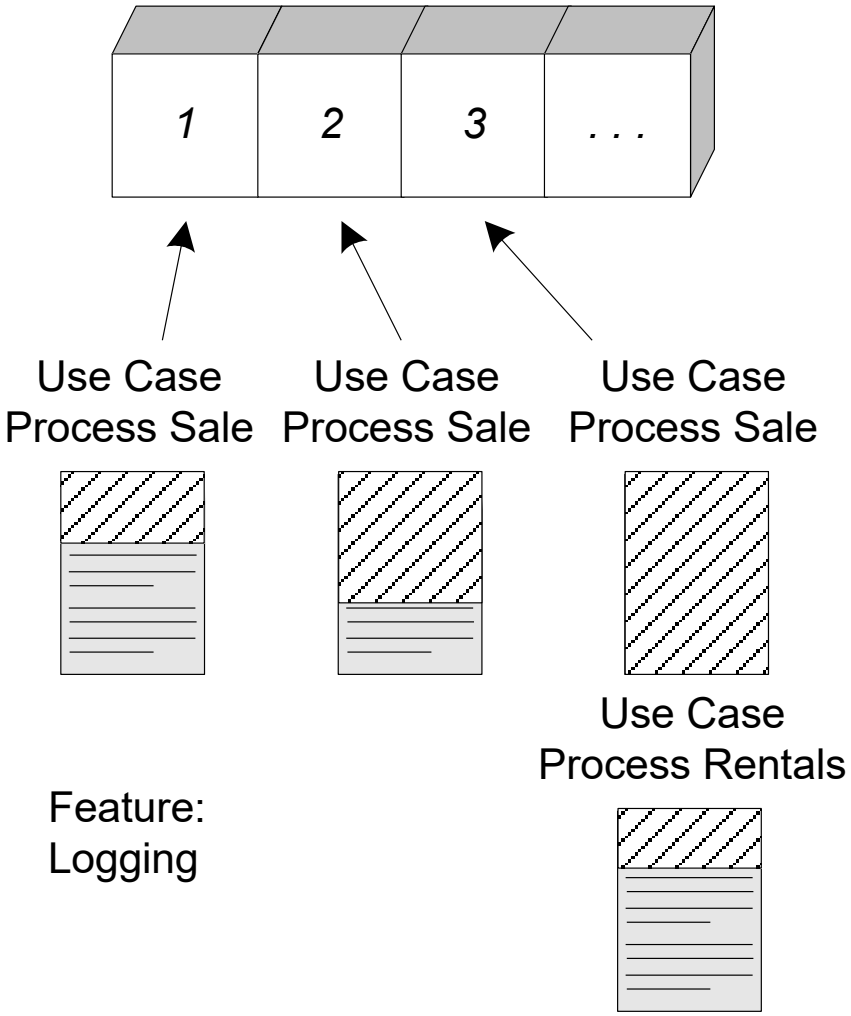
- Domain Model
- Design model
- Software Architecture Document
- Data Model
- Use-Case Story-boards, UI Prototypes

By the end of Elaboration, ~80% of the requirements are reliably defined

Requirements to Design—Iteratively

- Iteratively Do the Right Thing, Do the Thing Right
  - Requirements: do the right thing (c.f. validation)
  - Design: do the thing right (c.f. verification)
- Provoking (inevitable) Change Early
  - Although an iterative method embrace change, try to provoke the inevitable change in early iterations
  - Achieving a more stable goal, requirements discovery should stabilized
- Didn't All That Analysis & Modelling Take Weeks?
  - A few hours or days: use case writing, domain modelling
  - Mixed into a few weeks: proof-of-concept dev., finding resources, planning, setting up the environment, …

Spreading Use Cases Across Iterations



A use case or feature is often too complex to complete in one short iteration.

Therefore, different parts or scenarios must be allocated to different iterations.

How do Developers Design Objects?

1. **Code.** From mental model to code. Design-while-coding (Java, C#, ...), ideally with an IDE (e.g., Eclipse or Visual Studio) which supports refactoring and other high-level operations.
2. **Draw, then code.** Drawing UML on a whiteboard or UML CASE tool (e.g. EA), then switching to **1.**
3. **Only draw.** The tool generates everything from diagrams! *Many a dead tool vendor has washed onto the shores of this steep island.* "Only draw" is a misnomer; it still involves a programming language attached to the graphic elements.

If we use *Draw, then code* (the most popular approach with UML),
the **drawing overhead** should be worth the effort

Agile Modelling & Lightweight UML Drawing

- Modelling with other developers
- Static and dynamic models
  - Class *and* interaction diagrams
  - Create several models in parallel
- Hand draw, and/or
  - White boards, large surface area, digital capture
- UML tool
  - IDE integration, reverse or round trip (class and interaction diagrams)
- How long?: few hours to a day near iteration start (3wk iter.)

Object Design Skill vs UML Notation Skill

- Drawing UML is a reflection of making decisions about the design
- UML models should use correct notation as a principle of communication
  - However, of ***greatest importance*** is object design skill, not UML Notation skill
- Object design requires knowledge of
  - principles of responsibility assignment
  - design patterns

Summary

- In Unified Process (UP), Inception is the initial short step to establish a common vision and basic scope for the project
- Elaboration is the phase where software design and modelling is performed
  - The core architecture is programmed and tested (i.e., executable architecture)
- Translating requirements to design can be done iteratively

Lecture Identification

Coordinator: Dr Philip Dart

Lecturer: Dr Patanamon Thongtanunam

Semester 1, 2025

© University of Melbourne 2025

These slides include materials from:

Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, Third Edition, by Craig Larman, Pearson Education Inc., 2005.